

Intro to DevOps — LO4 & LO5 Exam Guide

This guide contains complete, exam-ready solutions and explanations for Learning Outcomes 4 and 5.

Learning Outcome 4 — 12 points / 30 min

Task 1 — LO4_M / 4 points

Requirement

Deploy a Deployment named `static-site-deployment` with **two replicas** using the `nginx:alpine` container image.

Command

```
kubectl create deployment static-site-deployment --image=nginx:alpine --replicas=2
```

Verify the Deployment

```
kubectl get deployments
```

You should see that `static-site-deployment` has two desired/ready replicas.

Also check the Pods:

```
kubectl get pods
```

For more detailed information:

```
kubectl describe deployment static-site-deployment
```

Explanation

A Kubernetes **Deployment** manages a set of Pods and their ReplicaSet. It describes the desired state of the application and Kubernetes continuously attempts to maintain that state.

In this task:

- `static-site-deployment` is the Deployment name.
- `nginx:alpine` is the container image.
- `--replicas=2` tells Kubernetes that two copies of the Pod should be running.

If one Pod crashes or is deleted, the Deployment/ReplicaSet automatically creates a replacement so that the desired number of replicas remains two.

Important correction

A command such as:

```
kubectl create deployment web --image=nginx:alpine --replicas=2
```

would technically create a working Deployment, but it would **not satisfy the question** because the required Deployment name is `static-site-deployment`, not `web`.

Short exam explanation

A Deployment named `static-site-deployment` is created using the `nginx:alpine` image with two replicas. The Deployment manages the underlying ReplicaSet and ensures that two nginx Pods remain running.

Task 2 — LO4_M / 4 points

Requirement

Create a ConfigMap named `site-content` containing a key called `index.html`. The value must contain a small HTML page with your **name, surname, and exam group**.

Mount that key into the nginx Deployment at:

```
/usr/share/nginx/html/index.html
```

using `subPath`, and verify the contents from inside a Pod using `kubectl exec`.

Step 1 — Create the ConfigMap

Replace the group below with your actual exam group if necessary.

```
kubectl create configmap site-content --from-literal=index.html='<html><body><h1>Luka Rokolj - Group A</h1></body></html>'
```

Verify the ConfigMap

```
kubectl get configmap site-content
```

For more information:

```
kubectl describe configmap site-content
```

You can also display it as YAML:

```
kubectl get configmap site-content -o yaml
```

What is a ConfigMap?

A Kubernetes **ConfigMap** stores non-sensitive configuration separately from the container image.

Examples include:

- configuration files
- environment variables
- URLs
- application settings
- HTML/static content

This means configuration can be changed without rebuilding the application image.

For passwords, API keys, and other sensitive information, a Kubernetes **Secret** should normally be used instead.

Step 2 — Mount the ConfigMap into nginx

Edit the Deployment:

```
kubectl edit deployment static-site-deployment
```

Inside the container definition add:

volumeMounts:

- name: site-content-volume
- mountPath: /usr/share/nginx/html/index.html
- subPath: index.html

Under the Pod **spec**, at the same structural level as **containers**, add:

volumes:

- name: site-content-volume

configMap:

name: site-content

The important section should resemble:

spec:

replicas: 2

selector:

matchLabels:

app: static-site-deployment

template:

metadata:

labels:

app: static-site-deployment

spec:

containers:

- name: nginx

image: nginx:alpine

volumeMounts:

- name: site-content-volume

mountPath: /usr/share/nginx/html/index.html

subPath: index.html

volumes:

- name: site-content-volume

configMap:

name: site-content

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: site-content
data:
  index.html: |
    <!DOCTYPE html>
    <html>
    <head>
      <title>Exam Static Site</title>
    </head>
    <body>
      <h1>Luka Rokolj</h1>
      <p>Exam group: Group A</p>
    </body>
    </html>
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: static-site-deployment
spec:
  replicas: 2

  selector:
    matchLabels:
      app: static-site

  template:
    metadata:
      labels:
        app: static-site

    spec:
      containers:
        - name: nginx
          image: nginx:alpine

      ports:
        - containerPort: 80

      volumeMounts:
        - name: site-content-volume
          mountPath: /usr/share/nginx/html/index.html
          subPath: index.html

      volumes:
```

- name: site-content-volume

configMap:
name: site-content

Note: the exact generated labels/container name can vary depending on how the Deployment was created. Preserve the existing selector and labels and add the volume configuration to the existing manifest.

Understanding **volumes** and **volumeMounts**

The **volumes** section tells Kubernetes **where the data comes from**:

volumes:

- name: site-content-volume

configMap:
name: site-content

This says that the volume called **site-content-volume** gets its contents from the ConfigMap named **site-content**.

The **volumeMounts** section tells Kubernetes **where that data appears inside the container**:

volumeMounts:

- name: site-content-volume

mountPath: /usr/share/nginx/html/index.html
subPath: index.html

So the relationship is:

ConfigMap: site-content

↓

Volume: site-content-volume

↓

Container volumeMount



/usr/share/nginx/html/index.html

Why is **subPath** required?

The ConfigMap contains a key named:

index.html

Using:

subPath: index.html

means Kubernetes mounts only that specific key/file at the requested path.

Without **subPath**, mounting the ConfigMap normally mounts it as a directory. Since the task specifically requires the key to be mounted as:

/usr/share/nginx/html/index.html

subPath allows us to replace that individual file instead of replacing the whole nginx HTML directory.

Easy way to remember

volume = where data comes from

volumeMount = where data goes in the container

subPath = mount one specific file/key

Step 3 — Check the Pods

After editing the Deployment, Kubernetes may perform a rollout because the Pod template has changed.

kubectl get pods

Wait until the new Pods show **Running** and **1/1** ready.

Step 4 — Verify the file with kubectl exec

First find a Pod name:

```
kubectl get pods
```

Example:

```
static-site-deployment-6f947f65c9-abc12
```

Then run:

```
kubectl exec static-site-deployment-6f947f65c9-abc12 -- cat /usr/share/nginx/html/index.html
```

Expected output:

```
<html><body><h1>Luka Rokolj - Group A</h1></body></html>
```

What does `kubectl exec` do?

`kubectl exec` executes a command inside a running container.

In this command:

```
kubectl exec POD_NAME -- cat /usr/share/nginx/html/index.html
```

- `kubectl exec` = execute something inside the Pod
- `POD_NAME` = target Pod
- `--` = separates kubectl arguments from the command executed inside the container
- `cat ...` = command executed inside the container

This proves that the ConfigMap file was successfully mounted.

Optional nginx verification

You can also request the webpage from nginx from inside the Pod:

```
kubectl exec POD_NAME -- wget -qO- localhost
```

It should return your HTML page.

Short exam explanation

The ConfigMap stores the custom `index.html` independently of the nginx image. A ConfigMap-backed volume is added to the Pod and the `index.html` key is mounted over nginx's default index page using `subPath`. `kubectl exec` and `cat` prove that the expected file exists inside the running container.

Task 3 — LO4_D / 4 points

Requirement

Update the Deployment from:

nginx:alpine

to:

nginx:stable-alpine

Then:

1. Follow the rolling update.
2. Show rollout history.
3. Roll back to the previous revision.
4. Prove that the final image is again `nginx:alpine`.

Step 1 — Check the current image

`kubectl describe deployment static-site-deployment`

Look for the image information.

A cleaner command is:

```
kubectl get deployment static-site-deployment -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected:

nginx:alpine

Step 2 — Update the image

Assuming the generated container is named `nginx`:

`kubectl set image deployment/static-site-deployment nginx=nginx:stable-alpine`

If unsure about the container name, inspect the Deployment:

```
kubectl get deployment static-site-deployment -o yaml
```

Find:

containers:

- name: nginx

The name before = in `kubectl set image` must match the actual container name.

General syntax:

```
kubectl set image deployment/DEPLOYMENT_NAME CONTAINER_NAME=NEW_IMAGE
```

Step 3 — Follow the rolling update

```
kubectl rollout status deployment/static-site-deployment
```

Expected successful result:

deployment "static-site-deployment" successfully rolled out

What is a rolling update?

A rolling update gradually replaces Pods running the old application version with Pods running the new version.

Instead of stopping all old Pods simultaneously, Kubernetes creates/replaces Pods progressively. This helps reduce or avoid application downtime.

Conceptually:

Old: nginx:alpine

↓

Create new Pod with nginx:stable-alpine

↓

Wait until new Pod is ready

↓

Remove an old Pod



Repeat

Deployments use the **RollingUpdate** strategy by default.

Step 4 — Show rollout history

```
kubectl rollout history deployment/static-site-deployment
```

Example output:

REVISION	CHANGE-CAUSE
----------	--------------

1	<none>
---	--------

2	<none>
---	--------

3	<none>
---	--------

Why might there be more than two revisions?

Changing the Pod template creates a new Deployment revision. Mounting the ConfigMap earlier may already have changed the Pod template, so the image update might not be revision 2.

This is normal.

kubectl rollout history allows administrators to inspect the Deployment's rollout revisions and supports reverting to an earlier state.

Step 5 — Roll back

```
kubectl rollout undo deployment/static-site-deployment
```

This rolls back to the immediately previous revision.

Check the rollback:

```
kubectl rollout status deployment/static-site-deployment
```

Step 6 — Prove the image is nginx:alpine

Using `describe`:

```
kubectl describe deployment static-site-deployment
```

Look for:

Image: nginx:alpine

Or use:

```
kubectl get deployment static-site-deployment -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected:

nginx:alpine

Short exam explanation

`kubectl set image` changes the container image in the Deployment's Pod template, triggering a rolling update. `kubectl rollout status` follows its progress and `kubectl rollout history` displays previous revisions. `kubectl rollout undo` restores the previous Deployment revision. Finally, `kubectl describe deployment` or JSONPath output proves that the image has returned to `nginx:alpine`.

LO4 Quick Command Sequence

1. Create Deployment

```
kubectl create deployment static-site-deployment --image=nginx:alpine --replicas=2
```

Verify

```
kubectl get deployments
```

```
kubectl get pods
```

2. Create ConfigMap

```
kubectl create configmap site-content --from-literal=index.html='<html><body><h1>Luka  
Rokolj - Group A</h1></body></html>'
```

```
# Verify ConfigMap
```

```
kubectl describe configmap site-content
```

```
# Edit Deployment and add volumes/volumeMounts
```

```
kubectl edit deployment static-site-deployment
```

```
# Check Pods
```

```
kubectl get pods
```

```
# Verify mounted file
```

```
kubectl exec POD_NAME -- cat /usr/share/nginx/html/index.html
```

```
# 3. Update image
```

```
kubectl set image deployment/static-site-deployment nginx=nginx:stable-alpine
```

```
# Follow rollout
```

```
kubectl rollout status deployment/static-site-deployment
```

```
# History
```

```
kubectl rollout history deployment/static-site-deployment
```

```
# Rollback
```

```
kubectl rollout undo deployment/static-site-deployment
```

```
# Follow rollback
```

```
kubectl rollout status deployment/static-site-deployment
```

```
# Prove image
```

```
kubectl describe deployment static-site-deployment
```

```
# OR
```

```
kubectl get deployment static-site-deployment -o  
jsonpath='{.spec.template.spec.containers[0].image}'
```

Learning Outcome 5 — 12 points / 30 min

Task 1 — LO5_M / 4 points

Given command

```
podman run --name mycontainer -d --port 8080:80 httpd
```

Problem

The port publishing option is incorrect.

The command uses:

--port

The correct Podman option is:

-p

or its long form:

--publish

Correct command

```
podman run --name mycontainer -d -p 8080:80 httpd
```

Equivalent long form:

```
podman run --name mycontainer -d --publish 8080:80 httpd
```

Understanding the complete command

```
podman run --name mycontainer -d -p 8080:80 httpd
```

podman run

Creates and starts a new container from an image.

--name mycontainer

Assigns the container the human-readable name `mycontainer`.

Without this option, Podman automatically generates a name.

-d

Means **detached mode**.

The container runs in the background and the terminal is returned to the user.

-p 8080:80

Publishes a container port to the host.

The format is:

HOST_PORT:CONTAINER_PORT

Therefore:

8080:80

means:

Host port 8080 → Container port 80

The Apache HTTP server inside the `httpd` container listens on port 80, but from the host it can be accessed using port 8080.

Conceptually:

Browser / curl

↓

localhost:8080

↓

Host port 8080

↓

Podman port mapping



Container port 80



Apache HTTP Server

httpd

This is the image used to create the container. It contains the Apache HTTP Server.

Verify

`podman ps`

You should see the running container and a port mapping similar to:

`0.0.0.0:8080->80/tcp`

Test it:

`curl localhost:8080`

Short exam answer

The command incorrectly uses `--port`. Podman uses `-p` or `--publish` to publish container ports. The corrected command is `podman run --name mycontainer -d -p 8080:80 httpd`. This runs the Apache container in detached mode and maps host port 8080 to port 80 inside the container.

Task 2 — LO5_M / 4 points

Given Dockerfile

Use the official Ubuntu base image

FROM ubuntu:latest

Set an environment variable

ENV student="your_value_here"

Install Nginx RUN yum update && \ yum install -y nginx && \ rm -rf /var/lib/apt/lists/*

Define the running port

EXPOSE 80

Start Nginx and keep it from running in background

CMD ["nginx", "-g", "daemon off;"]

There are multiple problems.

Error 1 — FRM is invalid

Wrong:

FRM ubuntu:latest

Correct:

FROM ubuntu:latest

Explanation

FROM specifies the base image used by the build.

Here:

FROM ubuntu:latest

means the new image starts from the latest Ubuntu base image.

FRM is simply not a valid Dockerfile instruction.

Error 2 — Ubuntu does not use yum

The broken installation command uses:

yum

But the selected base image is:

FROM ubuntu:latest

Ubuntu is Debian-based and normally uses the **APT** package management system.

Therefore use:

apt-get

instead of:

yum

A useful association to remember:

Ubuntu / Debian → apt / apt-get

RHEL / CentOS → yum / dnf

Error 3 — The RUN command is commented out

The line begins with #:

```
# Install Nginx RUN yum update ...
```

Anything after # on that line is a comment.

Therefore Docker does not execute RUN at all and nginx is never installed.

The comment and instruction must be separate:

```
# Install Nginx
```

```
RUN apt-get update && ...
```

Correct Dockerfile

```
FROM ubuntu:latest
```

```
ENV student="your_value_here"
```

```
RUN apt-get update && \
```

```
    apt-get install -y nginx && \
```

```
    rm -rf /var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Explanation of every instruction

FROM

```
FROM ubuntu:latest
```

Defines Ubuntu as the base image.

Every normal Dockerfile begins with a **FROM** instruction (apart from special parser directives/comments and certain advanced cases).

ENV

```
ENV student="your_value_here"
```

Creates an environment variable named:

student

inside the image/container.

It can later be accessed by applications or shell commands inside the container.

RUN

```
RUN apt-get update && \
```

```
    apt-get install -y nginx && \
```

```
rm -rf /var/lib/apt/lists/*
```

RUN executes commands **while the image is being built**.

apt-get update

Downloads/refreshes information about available packages.

apt-get install -y nginx

Installs nginx.

-y automatically answers yes to installation confirmation prompts, which is important for a non-interactive image build.

&&

Commands are chained using **&&**.

The next command executes only if the previous command succeeds.

**Backslash **

Allows the Dockerfile command to continue onto another line, improving readability.

rm -rf /var/lib/apt/lists/*

Deletes downloaded APT package-list data after installation.

This reduces unnecessary data in the resulting image layer and helps keep the image smaller.

EXPOSE

EXPOSE 80

Documents that the application inside the container is expected to listen on TCP port 80.

Important distinction

EXPOSE 80 does **not automatically publish port 80 to the host**.

You still normally publish the port when starting the container:

```
podman run -p 8080:80 IMAGE_NAME
```

Think of it as:

EXPOSE = image/container documentation

-p = actual host-to-container port publishing

CMD

CMD ["nginx", "-g", "daemon off;"]

Defines the default command executed when a container starts from the image.

Nginx normally supports daemon/background operation. In a container, the main service should remain in the foreground as the container's main process.

Therefore:

daemon off;

keeps nginx running in the foreground.

If the main process exits, the container stops.

Build and run the corrected Dockerfile

Assuming the file is called **Dockerfile**:

podman build -t my-nginx .

Then:

podman run -d -p 8080:80 my-nginx

Check:

podman ps

Test:

curl localhost:8080

Build time vs run time

This distinction is useful for exams:

RUN apt-get install ...

happens when building the **image**.

CMD ["nginx", ...]

runs when starting the **container**.

Remember:

Dockerfile --build--> Image --run--> Container

Short exam answer

The Dockerfile contains several errors. **FRM** is invalid and must be changed to **FROM**. The selected base image is Ubuntu, so nginx must be installed with **apt/apt-get** rather than **yum**. The installation command is also placed on a line beginning with **#**, making the entire line a comment, so it would never execute. The corrected Dockerfile uses **FROM ubuntu:latest** and a separate **RUN apt-get update && apt-get install -y nginx** instruction.

Task 3 — LO5_D / 4 points

Given Kubernetes Deployment

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

labels:

app: nginx

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: webserver
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

There are two important things to recognise.

Problem 1 — YAML indentation

YAML uses indentation to represent structure.

The manifest as displayed above is not properly indented. For example:

```
metadata:
  name: nginx-deployment
```

must have `name` nested under `metadata`.

Similarly, all other nested properties require correct indentation.

Incorrect indentation can make YAML invalid or change its structure.

Problem 2 — Deployment selector does not match Pod labels

This is the main Kubernetes logic error.

The selector says:

selector:

matchLabels:

app: nginx

This tells the Deployment:

The Pods belonging to this Deployment have the label `app=nginx`.

However, the Pod template creates Pods with:

template:

metadata:

labels:

app: webserver

So we have:

Deployment selector: `app=nginx`

Pod label: `app=webserver`

They do not match.

A Deployment's selector must match the labels on its Pod template.

Correct Kubernetes manifest

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

labels:

app: nginx

spec:

replicas: 1

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

Understanding the manifest

apiVersion

apiVersion: apps/v1

Specifies which Kubernetes API version is used for the resource.

Deployments use:

apps/v1

kind

kind: Deployment

Specifies the type of Kubernetes object being created.

In this case it is a Deployment.

metadata

metadata:

name: nginx-deployment

labels:

app: nginx

Contains identifying information about the Deployment.

name gives the resource its name.

labels are key-value metadata used to organise and select Kubernetes resources.

replicas

replicas: 1

Specifies the desired number of Pod replicas.

Kubernetes attempts to maintain exactly one Pod for this Deployment.

selector

selector:

matchLabels:

app: nginx

Tells the Deployment which Pods it manages.

It selects Pods carrying:

app=nginx

template

template:

Defines the template Kubernetes uses whenever it creates a new Pod for the Deployment.

You can think of it as a blueprint for the Pods.

Pod labels

template:

metadata:

labels:

app: nginx

The Pods generated by the Deployment receive the label:

app=nginx

This now matches:

selector:

matchLabels:

app: nginx

Why must they match?

The Deployment needs a reliable method to identify the Pods it owns/manages.

The selector says:

Find Pods where app=nginx

Therefore its template must create Pods carrying:

app=nginx

Correct:

Selector		Pod label
app=nginx	→	app=nginx

Incorrect:

Selector		Pod label
app=nginx	✗	app=webserver

With [apps/v1](#) Deployments, the selector and Pod template labels must be compatible; a mismatch causes validation failure rather than giving the Deployment a valid set of Pods to manage.

Containers

containers:

- name: nginx

image: nginx:latest

Defines the containers that run inside each Pod.

Here:

- container name = `nginx`
 - image = `nginx:latest`
-

containerPort

ports:

- containerPort: 80

Declares that the container application listens on port 80.

Important

Like Dockerfile `EXPOSE`, `containerPort` does not by itself expose the application outside the cluster.

To make a Deployment accessible through a stable network endpoint, a Kubernetes **Service** is normally used.

For example:

```
kubectl expose deployment nginx-deployment --port=80 --target-port=80
```

But a Service is **not required by this particular question** unless explicitly requested.

Apply the corrected manifest

Save it as:

deployment.yaml

Then:

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl get deployments
```

```
kubectl get pods
```

```
kubectl describe deployment nginx-deployment
```

You can also inspect labels:

```
kubectl get pods --show-labels
```

You should see:

```
app=nginx
```

Short exam answer

The Deployment's selector expects Pods with the label `app: nginx`, while the Pod template specifies `app: webserver`. The selector of a Deployment must match the labels of its Pod template. Changing the template label to `app: nginx` fixes the logical error. The YAML must also be correctly indented because YAML uses indentation to define its structure.

LO5 Quick Answers

Podman

Wrong

```
podman run --name mycontainer -d --port 8080:80 httpd
```

Correct

```
podman run --name mycontainer -d -p 8080:80 httpd
```

Remember

```
-p HOST:CONTAINER
```

```
-p 8080:80
```

Dockerfile

Main errors

FRM → FROM

yum → apt-get (because base image is Ubuntu)

... RUN → RUN must not be inside a comment

Correct

FROM ubuntu:latest

ENV student="your_value_here"

RUN apt-get update && \

apt-get install -y nginx && \

rm -rf /var/lib/apt/lists/*

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Kubernetes manifest

Wrong

selector:

matchLabels:

app: nginx

...

labels:

app: webserver

Correct

selector:

matchLabels:

app: nginx

...

template:

metadata:

labels:

app: nginx

Remember

Deployment selector == Pod template labels

Essential Concepts to Remember

Deployment

A Deployment manages ReplicaSets and Pods and maintains the application's desired state.

Deployment

↓

ReplicaSet

↓

Pods

Replica

A replica is another instance/copy of a Pod.

replicas: 2

means Kubernetes should maintain two Pods.

ConfigMap

Stores **non-sensitive configuration** outside the container image.

Configuration → ConfigMap

Passwords/API keys → Secret

Volume

Provides data/storage to a Pod.

In LO4, the volume obtains its data from a ConfigMap.

volumeMount

Defines where a volume is mounted inside a container.

subPath

Allows one particular file/key from a volume to be mounted at a specific file path.

Rolling Update

Gradually replaces old Pods with new Pods, helping maintain application availability during an update.

Rollback

Restores a previous Deployment revision.

kubectl rollout undo deployment/NAME

Podman -p

Publishes a container port to the host.

-p HOST_PORT:CONTAINER_PORT

Dockerfile FROM

Defines the base image.

FROM ubuntu:latest

Dockerfile RUN

Runs commands while building the image.

RUN apt-get update

Dockerfile CMD

Defines the default process started when a container runs.

CMD ["nginx", "-g", "daemon off;"]

EXPOSE vs -p

EXPOSE 80 → documents expected container port

-p 8080:80 → actually publishes/maps the port to the host

Kubernetes labels and selectors

Labels identify/group resources.

Selectors find resources carrying particular labels.

For a Deployment:

selector.matchLabels

MUST MATCH

Pod template metadata.labels

Last-Minute Cheat Sheet

LO4

```
kubectl create deployment static-site-deployment --image=nginx:alpine --replicas=2
```

```
kubectl create configmap site-content --from-literal=index.html='<html><body><h1>Luka  
Roklj - Group A</h1></body></html>'
```

```
kubectl edit deployment static-site-deployment
```

```
kubectl get pods
```

```
kubectl exec POD_NAME -- cat /usr/share/nginx/html/index.html
```

```
kubectl set image deployment/static-site-deployment nginx=nginx:stable-alpine
```

```
kubectl rollout status deployment/static-site-deployment
```

```
kubectl rollout history deployment/static-site-deployment
```

```
kubectl rollout undo deployment/static-site-deployment
```

```
kubectl rollout status deployment/static-site-deployment
```

```
kubectl describe deployment static-site-deployment
```

ConfigMap mount:

volumeMounts:

- name: site-content-volume

mountPath: /usr/share/nginx/html/index.html

subPath: index.html

volumes:

- name: site-content-volume

configMap:

- name: site-content

LO5

Podman

```
podman run --name mycontainer -d -p 8080:80 httpd
```

Remember:

-p 8080:80 = host 8080 → container 80

Dockerfile

```
FROM ubuntu:latest
```

```
ENV student="your_value_here"
```

```
RUN apt-get update && \
```

```
    apt-get install -y nginx && \
```

```
    rm -rf /var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

Remember:

FRM → FROM

Ubuntu → apt-get, not yum

RUN cannot be commented out

RUN = build time

CMD = container start/run time

EXPOSE ≠ publish

Kubernetes Deployment

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

labels:

app: nginx

spec:

replicas: 1

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx:latest

ports:

- containerPort: 80

Remember the most important rule:

`selector.matchLabels == template.metadata.labels`

Very Short Oral Answers

If asked to explain the tasks verbally, these are good compact answers.

What does a Deployment do?

A Deployment manages ReplicaSets and Pods and continuously maintains the desired state, including the requested number of replicas and application version.

What is a ConfigMap?

A ConfigMap stores non-sensitive configuration separately from the container image and can be exposed to containers as files or environment variables.

What does subPath do?

`subPath` allows me to mount one specific file from a volume rather than mounting the entire volume directory.

What is a rolling update?

A rolling update gradually replaces Pods running the old version with Pods running the new version, helping avoid downtime.

What does rollout undo do?

It rolls a Deployment back to its previous revision.

What does `-p 8080:80` mean?

It publishes container port 80 through port 8080 on the host. The first number is the host port and the second is the container port.

Why does Ubuntu use apt instead of yum?

Ubuntu is Debian-based and uses APT. Yum/DNF is associated with RHEL-family distributions.

RUN vs CMD?

`RUN` executes while the image is being built. `CMD` defines the default process that runs when a container starts.

EXPOSE vs -p?

EXPOSE documents the port expected to be used by the container, while **-p** actually publishes a container port on the host.

Why must selector and labels match?

A Deployment uses its selector to identify the Pods it manages, so its selector must match the labels assigned to the Pod template.

Final Memory Pattern

For LO4 remember:

CREATE → CONFIGURE → VERIFY → UPDATE → STATUS → HISTORY → UNDO → VERIFY

For LO5 remember:

Podman:

--port ❌

-p / --publish ✅

Dockerfile:

FRM ❌ → FROM ✅

yum ❌ → apt-get ✅

commented RUN ❌

Kubernetes:

selector app=nginx

↓ MUST MATCH

Pod label app=nginx

Kubernetes / Podman Exam Troubleshooting Cheat Sheet

1. First thing to do when something doesn't work

Always start with:

```
kubectl get pods
```

Then inspect the problematic Pod:

```
kubectl describe pod <pod-name>
```

Look at **Events at the bottom first.**

For logs:

```
kubectl logs <pod-name>
```

If there are multiple containers:

```
kubectl logs <pod-name> -c <container-name>
```

2. Check the Kubernetes cluster

```
kubectl get nodes
```

Expected:

STATUS

Ready

If using Minikube:

```
minikube status
```

Start it if necessary:

```
minikube start
```

Check Kubernetes:

kubectl cluster-info

3. See everything currently running

kubectl get all

More details:

kubectl get pods -o wide

kubectl get deployments

kubectl get replicaset

kubectl get services

kubectl get configmaps

4. Pod is Pending

Check:

kubectl describe pod <pod-name>

Common reasons:

- insufficient CPU/RAM
- node not Ready
- scheduling constraints
- volume problem
- image problem

Check nodes:

kubectl get nodes

5. ImageInspectError / ErrImagePull / ImagePullBackOff

Check:

```
kubectl describe pod <pod-name>
```

Look under **Events**.

Check which image Kubernetes is using:

```
kubectl get deployment static-site-deployment \
-o jsonpath='{.spec.template.spec.containers[*].image}'
```

Expected:

nginx:alpine

Correct the image:

```
kubectl set image deployment/static-site-deployment nginx=nginx:alpine
```

Restart:

```
kubectl rollout restart deployment static-site-deployment
```

Check:

```
kubectl get pods
```

Test pulling manually:

```
podman pull nginx:alpine
```

or:

```
docker pull nginx:alpine
```

6. CrashLoopBackOff

The container starts but keeps crashing.

Check logs:

```
kubectl logs <pod-name>
```

Check previous crashed container:

```
kubectl logs <pod-name> --previous
```

Then:

```
kubectl describe pod <pod-name>
```

Usually investigate:

- wrong command
 - application crash
 - missing configuration
 - missing file
 - incorrect volume
 - incorrect permissions
-

7. Deployment not creating Pods

```
kubectl get deployment
```

```
kubectl get replicaset
```

```
kubectl get pods
```

Then:

```
kubectl describe deployment static-site-deployment
```

Check replicas:

```
kubectl get deployment static-site-deployment
```

If replicas are 0:

```
kubectl scale deployment static-site-deployment --replicas=2
```

8. Stop / Start Deployment

Stop without deleting:

```
kubectl scale deployment static-site-deployment --replicas=0
```

Start again:

```
kubectl scale deployment static-site-deployment --replicas=2
```

Restart:

```
kubectl rollout restart deployment static-site-deployment
```

9. Delete broken Deployment and start again

Delete Deployment:

```
kubectl delete deployment static-site-deployment
```

Delete ConfigMap:

```
kubectl delete configmap site-content
```

If everything came from YAML:

```
kubectl delete -f static-site.yaml
```

Then recreate:

```
kubectl apply -f static-site.yaml
```

10. YAML won't apply

Run:

```
kubectl apply -f static-site.yaml
```

If there is an error, check:

- indentation
- spelling

- `apiVersion`
- `kind`
- required fields
- labels/selectors

You can validate without creating resources:

```
kubectl apply --dry-run=client -f static-site.yaml
```

Useful for catching YAML errors before deployment.

11. Check current YAML in Kubernetes

Deployment:

```
kubectl get deployment static-site-deployment -o yaml
```

ConfigMap:

```
kubectl get configmap site-content -o yaml
```

This is extremely useful if you're unsure what Kubernetes actually has configured.

12. Edit Deployment directly

```
kubectl edit deployment static-site-deployment
```

If Vim opens:

Start editing

Press:

i

Save and exit

Press:

Esc

Then:

:wq

Press Enter.

Exit WITHOUT saving

Esc

:q!

Press Enter.

13. Nano

Open:

```
nano static-site.yaml
```

Save:

Ctrl + O

Enter

Exit:

Ctrl + X

Then apply your changes:

```
kubectl apply -f static-site.yaml
```

14. ConfigMap problems

Check whether it exists:

```
kubectl get configmap
```

Inspect:

```
kubectl describe configmap site-content
```

Or:

```
kubectl get configmap site-content -o yaml
```

Expected concept:

data:

```
index.html: |
```

```
<html>
```

```
...
```

```
</html>
```

15. Check ConfigMap mounted file

Get Pod:

```
kubectl get pods
```

Then:

```
kubectl exec <pod-name> -- cat /usr/share/nginx/html/index.html
```

Expected: your HTML.

You can also enter the container:

```
kubectl exec -it <pod-name> -- /bin/sh
```

Inside:

```
ls /usr/share/nginx/html
```

```
cat /usr/share/nginx/html/index.html
```

Exit:

```
exit
```

16. Correct ConfigMap mount

Deployment should contain:

volumeMounts:

- name: site-content-volume
 - mountPath: /usr/share/nginx/html/index.html
 - subPath: index.html

And:

volumes:

- name: site-content-volume
 - configMap:
 - name: site-content

Remember:

`volumeMounts` goes inside the container.

`volumes` goes inside the Pod `spec`.

17. Selector / Label Error

These MUST match:

selector:

- matchLabels:
 - app: nginx

and:

template:

- metadata:

labels:

app: nginx

Wrong:

selector:

matchLabels:

app: nginx

labels:

app: webserver

Correct:

selector label = Pod label

18. Rolling Update

Change image:

kubectl set image deployment/static-site-deployment \

nginx=nginx:stable-alpine

Watch:

kubectl rollout status deployment/static-site-deployment

History:

kubectl rollout history deployment/static-site-deployment

Check image:

kubectl describe deployment static-site-deployment

19. Rolling update gets stuck

Check Pods:

```
kubectl get pods
```

Describe the new Pod:

```
kubectl describe pod <pod-name>
```

Check Deployment:

```
kubectl describe deployment static-site-deployment
```

If necessary:

```
kubectl rollout undo deployment/static-site-deployment
```

Then:

```
kubectl rollout status deployment/static-site-deployment
```

20. Rollback

```
kubectl rollout history deployment/static-site-deployment
```

Rollback:

```
kubectl rollout undo deployment/static-site-deployment
```

Verify:

```
kubectl rollout status deployment/static-site-deployment
```

Then:

```
kubectl describe deployment static-site-deployment
```

Or just show the image:

```
kubectl get deployment static-site-deployment \
-o jsonpath='{.spec.template.spec.containers[0].image}'
```

21. Podman container doesn't start

Check all containers:

```
podman ps -a
```

Logs:

```
podman logs <container-name>
```

Inspect:

```
podman inspect <container-name>
```

Remove broken container:

```
podman rm -f <container-name>
```

22. Podman port doesn't work

Correct:

```
podman run -d --name mycontainer -p 8080:80 httpd
```

NOT:

```
podman run --port 8080:80 httpd
```

Remember:

```
-p HOST:CONTAINER
```

Therefore:

```
-p 8080:80
```

means:

localhost:8080 → container:80

Check:

```
podman ps
```

Test:

curl localhost:8080

23. Dockerfile doesn't build

Build:

```
podman build -t myimage .
```

Check the error carefully.

Common mistakes:

FRM

 Wrong

FROM

 Correct

Ubuntu uses:

apt-get

not:

yum

Correct example:

```
FROM ubuntu:latest
```

```
ENV student="Luka"
```

```
RUN apt-get update && \
```

```
    apt-get install -y nginx && \
```

```
    rm -rf /var/lib/apt/lists/*
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

24. Test Dockerfile container

Build:

```
podman build -t exam-nginx .
```

Run:

```
podman run -d --name exam-nginx-container -p 8080:80 exam-nginx
```

Check:

```
podman ps
```

Test:

```
curl localhost:8080
```

Logs:

```
podman logs exam-nginx-container
```

25. "Already exists" error

If Kubernetes says a resource already exists, inspect it first:

```
kubectl get deployments
```

```
kubectl get configmaps
```

If using YAML, prefer:

```
kubectl apply -f static-site.yaml
```

rather than trying to **create** it again.

Or delete:

```
kubectl delete deployment static-site-deployment
```

```
kubectl delete configmap site-content
```

Then recreate.

26. Wrong namespace / can't find resource

Check:

```
kubectl get namespaces
```

Check current context:

```
kubectl config current-context
```

Search all namespaces:

```
kubectl get pods -A
```

```
kubectl get deployments -A
```

27. Fast "Something is Broken" Procedure

If you don't know what is wrong, follow this exact order:

```
kubectl get nodes
```

Then:

```
kubectl get pods
```

Then:

```
kubectl describe pod <broken-pod>
```

READ EVENTS AT THE BOTTOM.

Then:

```
kubectl logs <broken-pod>
```

Then:

```
kubectl describe deployment static-site-deployment
```

Then:

```
kubectl get deployment static-site-deployment -o yaml
```

This will solve or identify most exam problems.

28. Nuclear Option — Start Again

If you've completely broken the task and have enough time:

```
kubectl delete deployment static-site-deployment
```

```
kubectl delete configmap site-content
```

Check:

```
kubectl get all
```

```
kubectl get configmaps
```

Fix your YAML:

```
nano static-site.yaml
```

Validate:

```
kubectl apply --dry-run=client -f static-site.yaml
```

Then:

```
kubectl apply -f static-site.yaml
```

Check:

```
kubectl get pods
```

Then:

```
kubectl describe pod <pod-name>
```

29. Commands to Memorise

SEE WHAT IS HAPPENING

kubectl get pods

kubectl get all

kubectl describe pod <pod>

kubectl logs <pod>

DEPLOYMENT

kubectl get deployments

kubectl describe deployment <deployment>

kubectl get deployment <deployment> -o yaml

SCALE

kubectl scale deployment <deployment> --replicas=0

kubectl scale deployment <deployment> --replicas=2

RESTART

kubectl rollout restart deployment <deployment>

UPDATE

kubectl set image deployment/<deployment> nginx=nginx:stable-alpine

ROLLOUT

kubectl rollout status deployment/<deployment>

kubectl rollout history deployment/<deployment>

kubectl rollout undo deployment/<deployment>

YAML

kubectl apply --dry-run=client -f file.yaml

kubectl apply -f file.yaml

```
kubectl delete -f file.yaml
```

```
# CONFIGMAP
```

```
kubectl get configmaps
```

```
kubectl describe configmap site-content
```

```
# EXEC
```

```
kubectl exec <pod> -- cat /usr/share/nginx/html/index.html
```

```
kubectl exec -it <pod> -- /bin/sh
```

```
# PODMAN
```

```
podman ps
```

```
podman ps -a
```

```
podman logs <container>
```

```
podman inspect <container>
```

```
podman rm -f <container>
```

```
podman pull nginx:alpine
```



THE 5 COMMANDS TO REMEMBER IF YOU PANIC

1. What is broken?

```
kubectl get pods
```

2. Why is it broken?

```
kubectl describe pod <pod-name>
```

3. What did the application say?

```
kubectl logs <pod-name>
```

4. What configuration am I actually running?

```
kubectl get deployment static-site-deployment -o yaml
```

5. Completely restart the Deployment

```
kubectl rollout restart deployment static-site-deployment
```

Most Important Rule

When a Pod fails:

```
kubectl get pods
```

↓

```
kubectl describe pod <pod>
```

↓

READ EVENTS

↓

```
kubectl logs <pod>
```

↓

Fix configuration

↓

```
kubectl apply -f file.yaml
```

↓

```
kubectl get pods
```

Do not randomly change things. `kubectl describe pod` + `Events` usually tells you what Kubernetes is unhappy about.